

Reading App.tsx (2/3)

The Child Components

Where Are We?

GradeCalculatorApp	<- 4-3 (the state)
└ GradeInput	<- this part
└ GradeList	<- this part
└ GradeItem	<- this part
└ GradeResult	<- this part

Every component follows the same three sections:

Props interface → Business logic → JSX.

GradeInput Component

Interface

- An interface defines the shape (structure) of an object.
- In React, we use it to describe what props a component receives.

```
interface GradeInputProps {  
  onAdd: (grade: Omit<Grade, 'id'>) => string | null;  
}
```

means: "This component expects one prop, `onAdd`, and it must match this shape."

What are `onAdd` and `Omit<Grade, 'id'>`?

`onAdd` is a **callback function** that the parent passes in. It takes one grade and returns an error message (string), or `null` when there is no error.

A new grade does not have an id yet. The parent assigns it. `Omit` removes a field from a type:

```
Omit<Grade, 'id'>  
// means: { name: string; score: number; weight: number }
```

GradeInput Business Logic

1. Make states using **useState**

```
function GradeInput({ onAdd }: GradeInputProps) {  
  const [name, setName] = useState('');  
  const [score, setScore] = useState('');  
  const [weight, setWeight] = useState('');  
  const [error, setError] = useState('');
```

The first three states correspond to the three inputs; **error** holds a message to show the user. The inputs keep **strings** (what the user typed). We convert them to numbers on submit.

Grade Calculator (React Edition)

2. Make the handleSubmit function

- `e` is the event object for form submission.
`FormEvent` is a TypeScript type for it. Without it, TypeScript would not know that `e.preventDefault()` exists.
- We import it with the `type` keyword: `import { useState, useEffect, type FormEvent } from 'react'`. This means it is used only for type checking and disappears after compilation.
- By default, submitting a `<form>` reloads the page which would erase all your React state

```

const handleSubmit = (e: FormEvent) => {
  e.preventDefault(); // prevent the page reload
  setError('');

  const scoreNum = parseFloat(score);
  const weightNum = parseFloat(weight);

  // Validation (the weight check is the same as the score check)
  if (!name.trim()) {
    setError('Please enter an assignment name');
    return;
  }
  if (isNaN(scoreNum) || scoreNum < 0 || scoreNum > 100) {
    setError('Score must be between 0 and 100');
    return;
  }
  // ... the same check for weightNum

  // onAdd is given by the parent:
  // it returns an error message, or null on success
  const addError = onAdd({
    name: name.trim(), score: scoreNum, weight: weightNum
  });
  if (addError) { setError(addError); return; }

  // Clear form <- the UI updates automatically
  setName(''); setScore(''); setWeight('');
};

```

GradeInput JSX

- The input's value is `name`. When the user types, `setName` stores the typed text (`e.target.value`).
- This is a **controlled input**: React state is the single source of truth for the text.

```
<form onSubmit={handleSubmit} className="grade-form">
  <input type="text" value={name}
    onChange={(e) => { setError(''); setName(e.target.value); }}
    placeholder="Assignment name" />
  { /* the score and weight inputs follow the same pattern */ }
  <button type="submit">Add Grade</button>
</form>
{error && <div className="error">{error}</div>}
```

GradeList Component

GradeList interface and logic

```
interface GradeListProps {  
  grades: Grade[];  
  onDelete: (id: number) => void;  
}  
  
function GradeList({ grades, onDelete }: GradeListProps) {  
  if (grades.length === 0) {  
    return <p className="empty">No grades yet. Add one above!</p>;  
  }  
}
```

GradeList JSX

- For each grade, it generates a GradeItem component with `key`, `grade`, and `onDelete`.
- `key` lets React identify which item is which. Use a **stable id** (`grade.id`), not the array index.

```
return (  
  <div className="grade-list">  
    {grades.map(grade => (  
      <GradeItem  
        key={grade.id}  
        grade={grade}  
        onDelete={onDelete}    )  ))
```

GradelItem Component

Each item displays name, score, and weight. It also shows a button that calls the `onDelete` function with its own id.

```
interface GradeItemProps {
  grade: Grade;
  onDelete: (id: number) => void;
}

function GradeItem({ grade, onDelete }: GradeItemProps) {
  return (
    <div className="grade-item">
      <div className="grade-info">
        <strong>{grade.name}</strong>
        <span>{grade.score.toFixed(1)}%
          (Weight: {grade.weight.toFixed(1)}%)</span>
      </div>
      <button className="delete-btn"
        onClick={() => onDelete(grade.id)}>
        Delete
      </button>
    </div>
  );
}
```

GradeResult Component

Interface and Business Logic

The logic is only one call to the pure function. The component has no calculation of its own.

```
interface GradeResultProps {  
  grades: Grade[];  
}  
  
function GradeResult({ grades }: GradeResultProps) {  
  const summary = calculateFinalGrade(grades); // pure function  
  
  if (!summary) {  
    return <div className="result">Add grades to see your final grade</div>;  
  }  
  
  const letterGrade = getLetterGrade(summary.percentage);  
}
```

`summary` is `GradeSummary | null`, so TypeScript forces us to handle `null` before using `summary.percentage`.

GradeResult JSX

```
return (  
  <div className="result">  
    <h2>Final Grade: {summary.percentage.toFixed(2)}%</h2>  
    <div className={`letter-grade grade-${letterGrade}`}>  
      {letterGrade}  
    </div>  
    <small>  
      Total weight: {summary.totalWeight.toFixed(1)}%  
      {summary.totalWeight < MAX_WEIGHT - WEIGHT_EPSILON && (  
        <span>  
          <br />Note: Only {summary.totalWeight.toFixed(1)}%  
          of grades entered  
        </span>  
      )}  
    </small>  
  </div>  

```

`grade-${letterGrade}` becomes a CSS class
such as `grade-A` (a different color for each letter).

Key Points of This Part

- Every component has three sections: **Props interface** → **Business logic** → **JSX**.
- `GradeInput` keeps what the user typed in its own states (controlled inputs), and calls `onAdd` on submit.
- `GradeList` turns the array into items with `map`, using a stable `key`.
- `GradeItem` and `GradeResult` only display data. `GradeResult` calls a pure function and